МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федерального государственного бюджетного образовательного учреждения
высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»
Техникум Пермского института (филиала)

Практическая работа №2
по дисциплине «Поддержка и тестирование программных модулей»

Руководитель _____ /Д.Б. Берестов/
«_20_» _марта_ 2026 г.
Исполнитель _____ /А.А. Визгалова/
«_20_» _марта_ 2026 г.

Пермь 2026 г.

Оглавление

Задание 3
Что такое модульные тесты и зачем они нужны? 3
Как устроен наш проект? 3
Создание проекта в Visual Studio 5
Шаг 1. Открытие Visual Studio 5
Шаг 2. Разработка интерфейса (XAML) 5
Шаг 3. Создание и написание Библиотеки Классов 6
Шаг 4. Подключение библиотеки к интерфейсу 7
Шаг 5. Создание и написание модульных тестов 8
Шаг 6: Построение решения 10
Шаг 7. Написание логики (C#) 11
Шаг 8. Написание кода для консоли и подключение библиотеки. 11
Заключение 12

Задание

Создать калькулятор с интерфейсом WPF. Приложение должно выполнять арифметические вычисления с операциями +, -, *, /, ^

Структура проекта:

1. WPF-приложение vizgalova_calculator_2pr
2. Библиотека классов vizgalova_library1
3. Модульные тесты vizgalova_tests
4. Консольное приложение ConsoleCalculator.

Что такое модульные тесты и зачем они нужны?

Модульные тесты - это маленькие программы, которые проверяют, правильно ли работают отдельные кусочки кода. Допустим, есть функция сложения. Тест подаёт ей 2 и 3 и ждёт, что получится 5. Если так и есть - тест пройден, если нет - он падает.

Зачем это нужно? Во-первых, чтобы быстро ловить ошибки. Во-вторых, если потом кто-то захочет что-то поменять в коде, тесты сразу покажут, не сломалось ли то, что раньше работало.

Как устроен наш проект?

У нас всё разбито на три части, чтобы было удобно:

1. Библиотека классов - это просто код с вычислениями (сложение, деление и т.д.). Никаких кнопок и окошек, только математика. Её удобно проверять тестами и можно использовать в других проектах.

2. WPF-приложение - это то, что видит пользователь: окошки, кнопки, поля для ввода. Оно вызывает методы из библиотеки и показывает результат. Само ничего не считает.

3. Модульные тесты - здесь лежат все проверки для библиотеки. Например, проверяем, что 2+2=4, а при делении на ноль всё падает с ошибкой, как и задумано.

Создание проекта в Visual Studio

Шаг 1. Открытие Visual Studio

Открыть Visual Studio, где виднеется главный экран. Выбрать «Создание проекта». Дальше выбрать WPF-приложение (.NET Framework), назвав его vizgalova_calculator_2pr. Выбираю платформу .NET Framework 4.7.2.

Шаг 2. Разработка интерфейса (XAML)

В файле MainWindow.xaml я создала интерфейс калькулятора. Окно размером 309x400 открывается по центру экрана. Фон сделала голубоватым.

Для ввода я добавила два поля: ResultText (крупный шрифт) для результата и ExpressionText для вводимого выражения. Они расположены в рамке с синей окантовкой.

Кнопки расположены сеткой 5×4. В первом ряду: скобки, степень и деление. Дальше идут цифры от 0 до 9 и операторы (умножение, вычитание, сложение). Отдельно есть кнопка с точкой для десятичных дробей, кнопка C для очистки и кнопка "=", которая занимает одну колонку.

Для всех кнопок я задала единый стиль: при наведении мышки кнопка меняет цвет - это помогает понять, что она нажалась.

Вид калькулятора можно увидеть на рисунке 1.

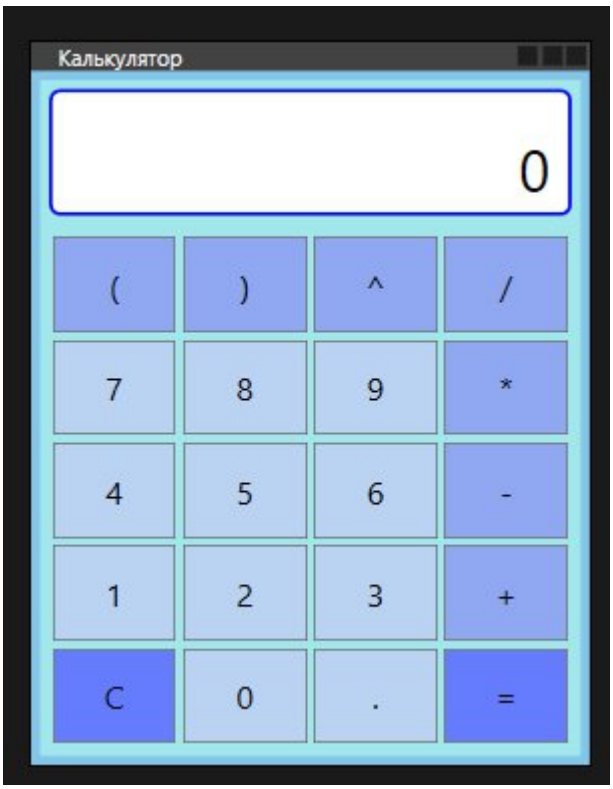

Рис. 1 – Интерфейс калькулятора

Шаг 3. Создание и написание Библиотеки Классов

Я открыла Visual Studio и нажала «Создать новый проект». Выбрала шаблон «Библиотека классов» (.NET Framework). Назвала проект vizgalova_library1, а решение Cals. Версия .NET – 4.7.2.

Выбор нужного проекта можно увидеть на рисунке 2.

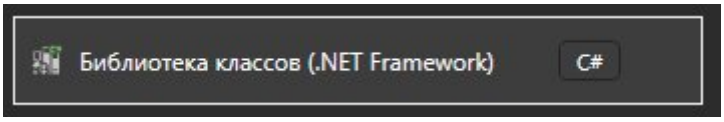

Рис 2 – Выбор проекта

В библиотеке vizgalova_library1 я реализовала основные операции: сложение, вычитание, умножение, деление и возведение в степень.

Главный метод Calculate принимает строку с выражением, заменяет запятую на точку для работы с десятичными дробями и вычисляет содержимое скобок. Затем выражение передаётся в метод SimpleCalculate, который

разбивает его на числа и операторы и выполняет действия в правильном порядке: сначала степень, потом умножение и деление, затем сложение и вычитание.

Для преобразования строк в числа используется CultureInfo.InvariantCulture, чтобы программа работала независимо от настроек системы.

Все вычисления используют базовые методы Add, Sub, Mul, Div, Pow. Метод Execute выполняет одну операцию по заданному оператору.

После написания кода я нажала «Построить решение», чтобы убедиться, что ошибок нет. Всё собралось успешно.

Шаг 4. Подключение библиотеки к интерфейсу

В проекте vizgalova_calculator_2pr добавила ссылку на vizgalova_library1: правой кнопкой на «Зависимости» => «Добавить ссылку на проект». Это показано на рисунке 3.

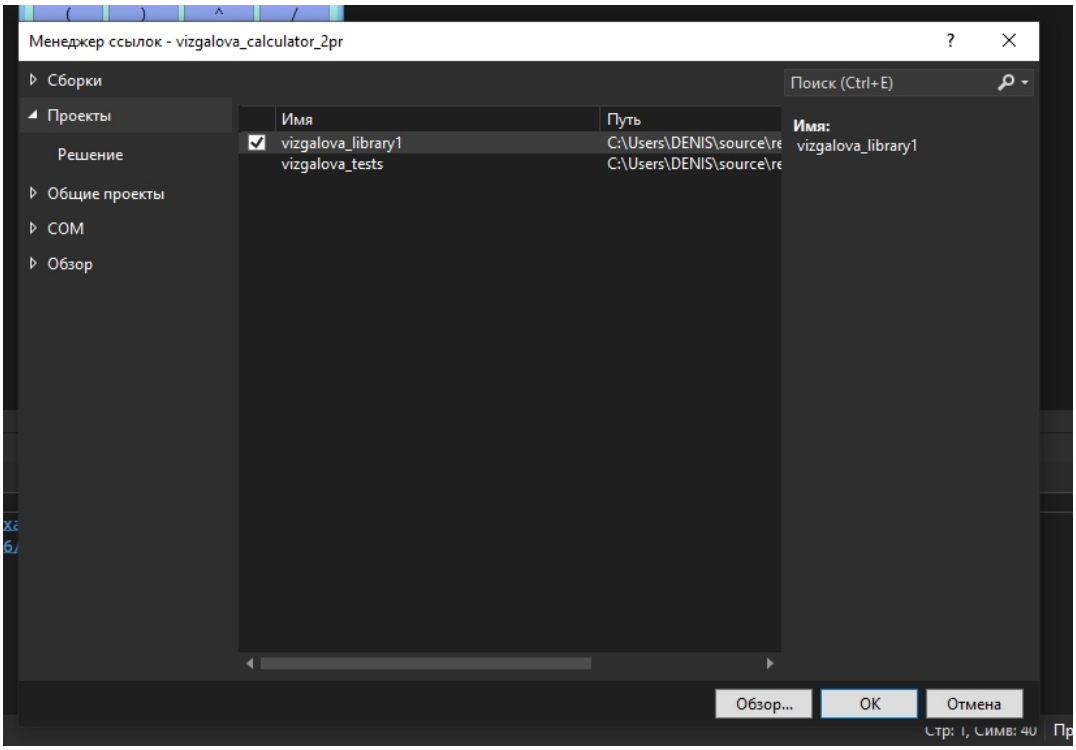

Рис. 3 – Подключение библиотеки

Шаг 5. Создание и написание модульных тестов

Дальше нужно было добавить проект для тестов. Я нажала правой кнопкой на решении, выбрала «Добавить» --> «Создать проект». В поиске ввела «test» и выбрала «Тестовый проект MSTest (Майкрософт)». Назвала его vizgalova_tests, а решение CalculatorTests.

Выбор нужного проекта можно увидеть на рисунке 4.

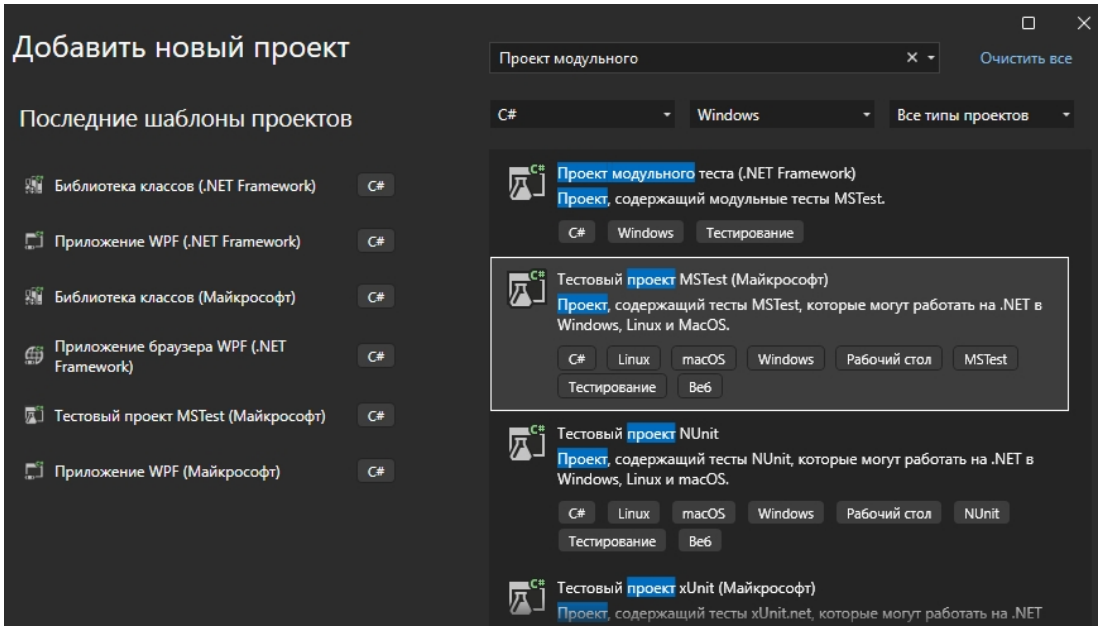

Рис 4 - Создание проекта тестов

Далее надо подключить библиотеку к тестам. В проекте vizgalova_tests нажала правой кнопкой на «Зависимости» --> «Добавить ссылку на проект» и выбрала vizgalova_library1.

Окно зависимостей можно увидеть на рисунке 5.

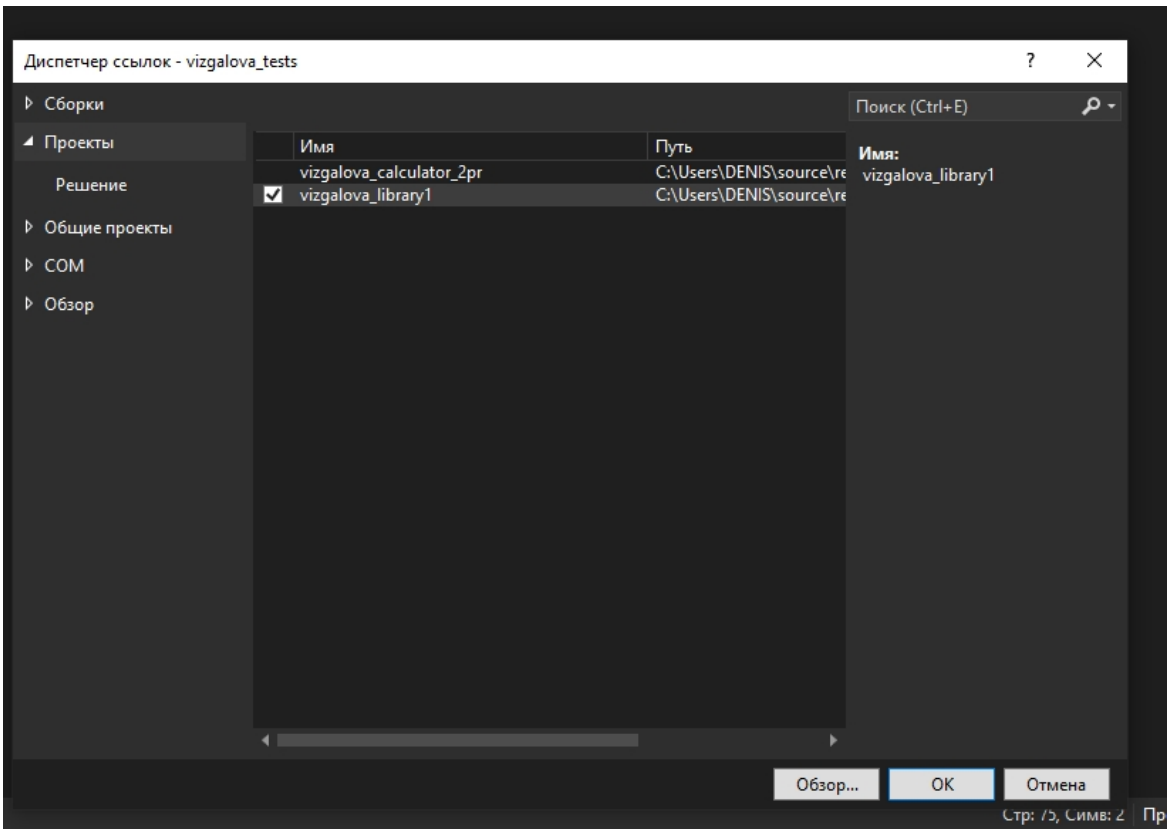

Рис 5 - Подключение библиотеки

Для проверки работоспособности библиотеки я написала модульные тесты. Каждый тест проверяет отдельный метод.

Я протестировала все базовые операции:

1. Add — сложение (2+3=5)
2. Sub — вычитание (10-2=8)
3. Mul — умножение (2×6=12)
4. Div — деление (15÷5=3)
5. Pow — возведение в степень (2^3=8)

Для метода Calculate, который вычисляет сложные выражения, я написала несколько тестов:

1. простое сложение (2+2=4)
2. приоритет операций (2+2×2=6)
3. выражения со скобками ((2+2)×2=8)
4. возведение в степень (2^3+1=9)

5. сложное выражение ((2+3)×4-1=19)

Далее я провела тестирование исключений.

Отдельно я проверила обработку деления на ноль. В тесте Div_by0_throwsException я вызываю метод Div с аргументами (10, 0) и проверяю, что выбрасывается исключение DivideByZeroException с правильным сообщением.

Все тесты успешно проходят, что подтверждает корректную работу библиотеки.

Шаг 6: Построение решения

В Visual Studio открыть «Обозреватель решений», в верхнем меню выбрать Сборка --> Пересобрать решение. Visual Studio скомпилирует все проекты. Далее нажать на «Тест» --> «Обозреватель тестов» На 6 рисунке показано проведение тестирования. После внесения исправлений все тесты были запущены повторно. Все они успешно прошли проверку.

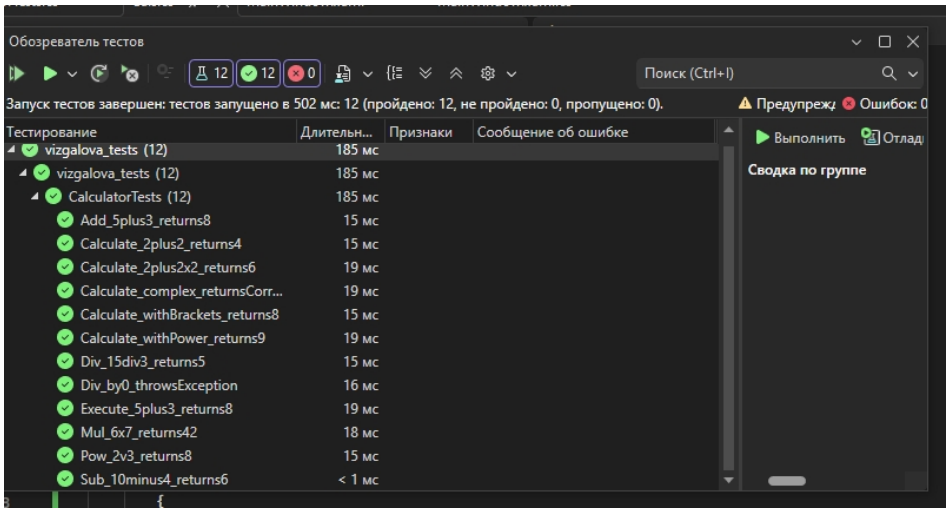

Рис. 6 - Результаты тестов

Шаг 7. Написание логики (C#)

В файле MainWindow.xaml.cs я написала логику калькулятора.

Все кнопки используют один обработчик Button_Click. При нажатии цифры или операторы добавляются к строке выражения s. Я добавила защиту: нельзя ввести два оператора подряд или две точки.

Кнопка "C" очищает всё.

При нажатии "=" вызывается метод Calculate из библиотеки vizgalova_library1. Результат выводится на экран, а выражение сохраняется для продолжения расчётов.

При делении на ноль программа показывает сообщение и выводит знак бесконечности. Другие ошибки (незакрытые скобки и т.п.) тоже обрабатываются.

Шаг 8. Написание кода для консоли и подключение библиотеки.

Я открыла Visual Studio и нажала «Создать новый проект». Выбрала

шаблон «Консольное приложение» (.NET Framework). Назвала проект ConsoleCalculator, а решение оставила Program. Версия .NET – 4.7.2.

После написания кода я подключила библиотеку как показано на рисунке 7.

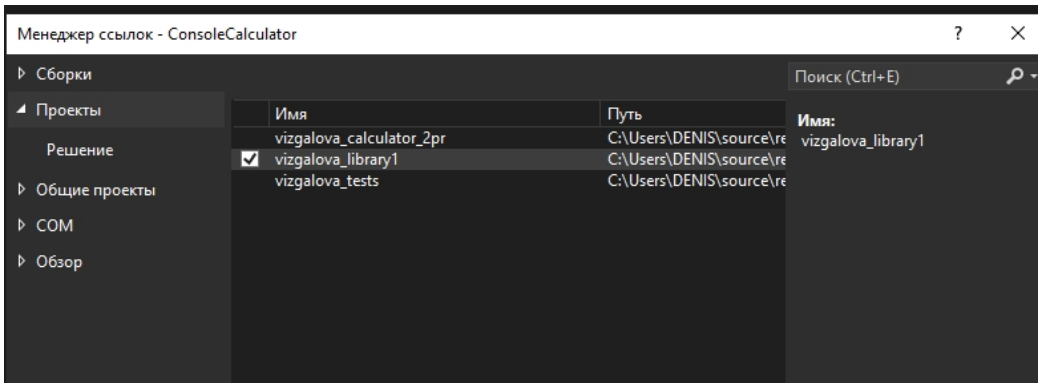

Рис. 7 – Добавление библиотеки

Чтобы запустить проект через консоль нажимаем на ConsoleCalculator в обозревателе решений правой кнопкой мыши --> «Назначить в качестве запускаемого проекта», после нажимаем F5.

На рисунке 8 показан пример работы калькулятора через консоль.

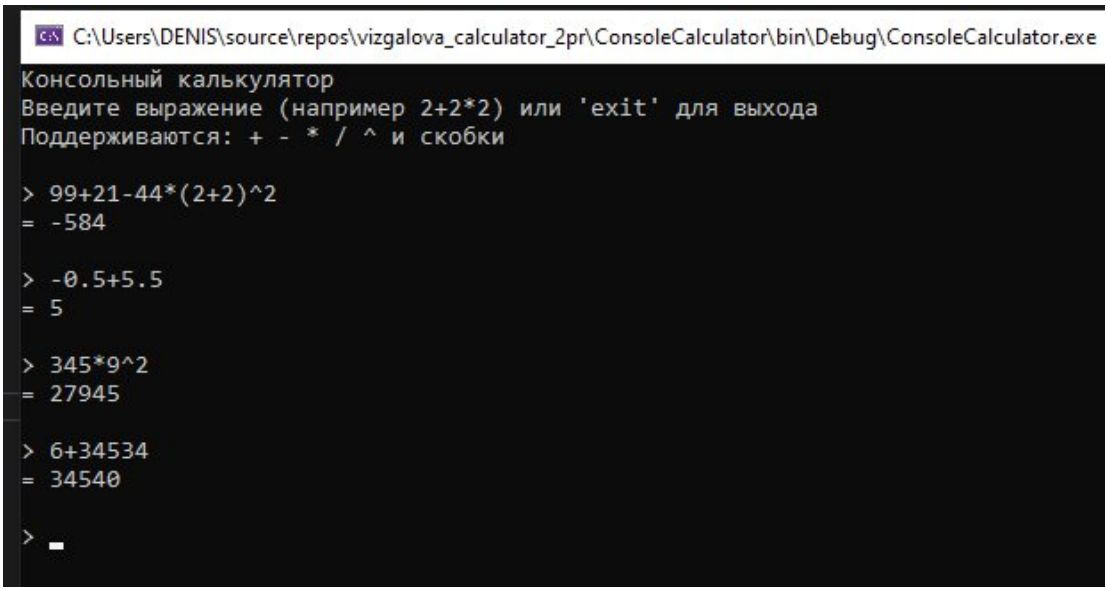

Рис. 8 – Калькулятор в консоли

Заключение

В ходе работы я сделала полноценный калькулятор на WPF. Всю логику вычислений я вынесла в отдельную библиотеку, чтобы её можно было использовать в разных проектах. Для проверки библиотеки я написала модульные тесты, все они успешно проходят.

Мой калькулятор умеет складывать, вычитать, умножать, делить и возводить в степень. Он правильно понимает порядок действий и работает со скобками. Я добавила защиту от неверного ввода: нельзя поставить два оператора подряд или две точки. При делении на ноль программа показывает "∞" и сообщение об ошибке.

Чтобы показать, зачем нужна библиотека, я сделала ещё и консольную версию калькулятора. Она использует ту же библиотеку и работает так же правильно, как и обычный калькулятор с кнопками.

За время работы я научилась:

1. создавать несколько проектов в одном решении;
2. подключать библиотеки к WPF и к тестам;
3. писать модульные тесты и разбираться в их результатах;
4. разбирать выражения со скобками и степенью;
5. защищать программу от неправильного ввода.

Все тесты пройдены, калькулятор работает без ошибок. Я довольна результатом.